



SAPIENZA
UNIVERSITÀ DI ROMA

Centralized ILP Orchestrator for Task Routing in Satellite Networks

Faculty of Information Engineering, Computer Science and Statistics
Computer Science

Claudio Bagini

ID number 2045337

Advisor

Dr. Emiliano Casalicchio

Co-Advisor

Academic Year 2025/2026

Thesis not yet defended

Centralized ILP Orchestrator for Task Routing in Satellite Networks

Bachelor's Thesis. Sapienza University of Rome

© 2026 Claudio Bagini. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Version: August 9, 2026

Author's email: bagini.2045337@studenti.uniroma1.it

Dedication

Abstract

The evolution of Low Earth Orbit (LEO) satellite megaconstellations has opened new perspectives for Satellite Computing, enabling the extension of edge computing capabilities directly into space (Orbital Edge Computing). However, the highly dynamic network topology, stringent energy constraints of satellite nodes, and strict deadlines of real-time applications make task orchestration and offloading critical challenges.

This thesis presents the design, implementation, and performance evaluation of a centralized orchestrator based on Integer Linear Programming (ILP). To mitigate solver computational costs and prevent starvation, the system integrates a dynamic batching mechanism controlled by Batch Size (BS) and Batch Timeout (BT) metrics, along with a coordinated formulation of timing constraints that accounts for queuing delay prior to resolution.

Using discrete-event simulations developed with the SimPy framework, the architecture was evaluated under stress across varying request arrival rates (λ), comparing *Mapping Time* and *Mapping Energy* optimization strategies. A detailed analysis revealed that system collapse under heavy loads ($\lambda > 6$ req/s) is predominantly driven by timeout deadlines (*Deadline Exceeded*) rather than physical compute resource saturation. Furthermore, sensitivity experiments demonstrated the effectiveness of deadline relaxation (ΔD) in recovering high completion rates.

Finally, the centralized orchestrator was benchmarked against state-of-the-art distributed and heuristic solutions (*OrbitAware*, *DTS-base*, *DTS-APopt*, and distributed ILP). The results quantify the fundamental trade-off between the theoretical global optimality of centralized allocation and the impact of queuing latency compared to the responsiveness of distributed models.

Acknowledgments

Acknowledgments

Contents

1	Introduction	1
1.1	Motivation and Context	1
1.2	Problem Statement	2
1.3	Objectives and Main Contributions	3
1.4	Thesis Outline	4
2	Background and State of the Art	5
2.1	Satellite Computing Paradigms	5
2.2	Task Offloading and Routing in LEO Networks	6
2.2.1	System Entities and Network Topology	6
2.2.2	Workload Profiles and Constraints	7
2.3	Centralized vs. Distributed Orchestration	7
2.3.1	Centralized Orchestration	7
2.3.2	Distributed Orchestration	7
2.4	State-of-the-Art Approaches	8
2.4.1	Distributed Heuristic Frameworks	8
2.4.2	Optimization-Based and Distributed ILP Formulations	8
2.5	Research Gap and Thesis Positioning	9
3	System Architecture and ILP Formulation	10
3.1	System Model and Assumptions	10
3.2	ILP Mathematical Formulation	10
3.2.1	Decision Variables and Objective Functions	10
3.2.2	System Constraints	10
3.3	Dynamic Batching Mechanism	11
3.4	Timing and Queuing Delay Compensation	11
4	Performance Analysis and Evaluation	12
4.1	Simulation Framework	12
4.2	Experimental Setup and Workload Profiles	12
4.3	Evaluation of Optimization Strategies	12
4.3.1	Mapping Time vs. Mapping Energy Comparison	12
4.3.2	Impact of Batching Parameters (<i>BS</i> and <i>BT</i>)	12
4.4	Bottleneck and Collapse Analysis	13
4.4.1	System Behavior Under Heavy Load ($\lambda > 6$ req/s)	13
4.4.2	Physical Saturation vs. Deadline Exceeded Failures	13

4.5	Sensitivity Analysis: Deadline Relaxation (ΔD)	13
5	Comparative Benchmark	14
5.1	Experimental Setup and Baseline Algorithms	14
5.2	Comparative Performance Evaluation	14
5.3	Trade-Off Analysis	14
6	Conclusions and Future Work	15
6.1	Summary of Findings	15
6.2	Limitations of the Centralized Approach	15
6.3	Future Research Directions	15
	Bibliography	16

Chapter 1

Introduction

In recent years, space communications and satellite networks have undergone a monumental paradigm shift. The rapid miniaturization of space components, combined with drastically reduced launch costs, has catalyzed the deployment of Mega-Constellations operating in Low Earth Orbit (LEO). Traditionally, satellite architectures functioned strictly under a "bent-pipe" model, where satellites served as passive transparent relays forwarding raw telemetry or communication signals back to Ground Stations (GS) for processing. However, the unprecedented volume of data produced by modern spaceborne applications—such as high-resolution Earth Observation (EO), environmental monitoring, and ubiquitous Internet of Things (IoT) sensing—has rendered this traditional pipeline highly inefficient.

To overcome the severe bandwidth limits of space-to-ground downlinks and the high propagation delays associated with remote terrestrial processing, the paradigm of **Orbital Edge Computing (OEC)**—also broadly termed Satellite Computing (SC)—has emerged. By equipping LEO satellites with onboard processing hardware, data can be computed directly in space close to the source, transmitting only actionable insights or highly compressed results to ground users. Despite its immense potential, executing compute-intensive tasks on resource-constrained satellite nodes introduces complex orchestration challenges that require advanced decision-making frameworks.

1.1 Motivation and Context

The transition toward Orbital Edge Computing is driven by two main technological trends: the exponential increase in space-generated data and the strict temporal requirements of next-generation applications. Applications such as real-time disaster response, autonomous maritime navigation, and defense surveillance demand sub-second latency for task execution and reporting. Processing these tasks on Earth introduces unavoidable latency bottlenecks due to intermittent Ground Station contact windows, dynamic weather interference, and severe downlink congestion.

LEO megaconstellations (such as Starlink, OneWeb, and Kuiper) consist of hundreds to thousands of satellites orbiting at altitudes between 300 km and 2000 km. These satellites move at high orbital velocities (approximately 7.5 km/s relative to the Earth's surface), resulting in a continuously changing network topology. Inter-

Satellite Links (ISLs)—utilizing high-throughput Free-Space Optics (FSO) or Radio Frequency (RF) channels—enable direct communication between adjacent satellites without routing signals down to the ground.

Consequently, a LEO constellation effectively forms a distributed, high-velocity edge computing network in orbit. However, executing applications across this orbital fabric requires intelligent **Task Offloading and Routing Orchestration**. Ground users or source satellites generate computational tasks that must be dynamically assigned to suitable processing nodes across the constellation. Deciding *where* and *when* to execute a given task, while routing its associated payload across multiple dynamic ISL hops, constitutes a high-dimensional optimization problem that directly governs overall network performance.

1.2 Problem Statement

Orchestrating task offloading in Orbital Edge Computing environments presents a unique set of constraints and operational challenges that set it apart from conventional terrestrial cloud/edge systems:

- **Severe Energy Budget Limits:** Satellite Edge Nodes (SENs) rely on solar panels and onboard battery storage. Computationally intensive task execution and RF/optical transmissions consume substantial power. Over-utilizing specific nodes leads to rapid battery depletion, thermal overhead, or total operational failure.
- **Heterogeneous and Limited Hardware Resources:** Onboard processing units (e.g., embedded CPUs, specialized AI accelerators) are constrained by strict Space, Weight, and Power (SWaP) requirements, as well as radiation-hardening constraints.
- **Dynamic Topology and Intermittent Connectivity:** High orbital mobility leads to frequent topology reconfigurations, changing link distances, and periodic loss of line-of-sight. Task routing paths must be continuously recalculated to avoid dropped links.
- **Strict Temporal Constraints and Queuing Delays:** Tasks possess strict deadlines (D_r). If the sum of transmission latency, queuing delay, and computation time exceeds D_r , the task fails, resulting in wasted system energy and degraded quality of service.

To achieve theoretical optimality in task placement and energy management, mathematical optimization tools such as **Integer Linear Programming (ILP)** are commonly employed. An ILP formulation can search the joint solution space of task routing, CPU allocation, and energy minimization to find globally optimal schedules.

However, centralized ILP solvers suffer from inherent computational complexity (NP-hard). In dynamic satellite networks with continuously arriving workloads, solving an ILP instance for every individual task request introduces unacceptable

computational overhead and processing latency at the central orchestrator. Conversely, accumulating task requests to solve them in batches introduces a trade-off: larger batches allow the ILP solver to achieve superior global resource allocation, but prolonged waiting times in orchestrator queues increase the risk of task deadline expiration (*Deadline Exceeded* failures).

1.3 Objectives and Main Contributions

The main objective of this thesis is to design, implement, and rigorously evaluate a **Centralized ILP Orchestration Framework** optimized for task routing and resource allocation in LEO satellite networks, incorporating a **Dynamic Batching Mechanism** to balance solver computational complexity with network queuing delay.

Specifically, this work addresses the key question: *To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?*

The main contributions of this thesis are summarized as follows:

1. **Formalization of the Centralized ILP Optimization Model:** We formulate a comprehensive ILP mathematical model that jointly optimizes task routing across Inter-Satellite Links and computational resource allocation on Satellite Edge Nodes. The model supports dynamic weighting between latency optimization (*Mapping Time*) and total energy preservation (*Mapping Energy*).
2. **Design of an Integrated Dynamic Batching & Timeout Mechanism:** To mitigate the computational overhead of solving the ILP model continuously, we integrate an adaptive batching policy governed by Batch Size (*BS*) and Batch Timeout (*BT*). Furthermore, we propose a mathematical deadline compensation formula to adjust task deadlines based on orchestrator queuing duration (T_{batch}).
3. **Development of a Discrete-Event Simulation Environment:** We construct a custom, high-fidelity discrete-event simulation framework in Python using *SimPy*. The simulator accurately models request arrival dynamics (λ), physical network links, queue management, hardware processing, and energy consumption under varied workload conditions.
4. **In-Depth Performance and Bottleneck Analysis:** We perform extensive stress testing under high workload arrival rates ($\lambda > 6$ req/s) to identify system collapse thresholds, distinguishing between physical resource saturation (CPU/Network capacity) and temporal violations (*Deadline Exceeded*). We also conduct sensitivity analyses regarding deadline relaxation (ΔD).
5. **Comparative Benchmark Against State-of-the-Art Solutions:** We evaluate the centralized ILP orchestrator against established baseline heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP formulations from the literature, establishing clear operational trade-offs regarding task completion rate, overall latency, and energy efficiency.

1.4 Thesis Outline

The remainder of this thesis is structured as follows:

- **Chapter 2 (Background and State of the Art):** Reviews the fundamental concepts of Satellite Computing, LEO network dynamics, and existing literature on task offloading. It provides a taxonomy of centralized versus distributed orchestration schemes and details key reference algorithms.
- **Chapter 3 (System Architecture and ILP Formulation):** Presents the formal mathematical system model, decision variables, objective functions (*Mapping Time* vs. *Mapping Energy*), constraints, and the dynamic batching/timeout mechanisms.
- **Chapter 4 (Performance Analysis and Evaluation):** Describes the SimPy discrete-event experimental setup and evaluates the proposed centralized orchestrator under varying batching configurations, workload profiles, heavy-load conditions, and deadline relaxation scenarios.
- **Chapter 5 (Comparative Benchmark):** Details the comparative analysis benchmarking our centralized orchestrator against state-of-the-art distributed heuristics and formulations, highlighting key trade-offs in latencies, energy consumption, and global optimality.
- **Chapter 6 (Conclusions and Future Work):** Summarizes the main findings of this research, outlines limitations of centralized orbital orchestration, and proposes future research directions toward hybrid adaptive architectures.

Chapter 2

Background and State of the Art

The rapid proliferation of Low Earth Orbit (LEO) satellite constellations has fundamentally redefined the landscape of satellite communications and space-borne computing. Transitioning from traditional passive data relays to active processing nodes requires a comprehensive understanding of orbital dynamics, task offloading models, and orchestration strategies. This chapter provides the theoretical foundation and contextual literature for this thesis. It details the paradigms of Orbital Edge Computing (OEC), analyzes task offloading and routing dynamics in LEO networks, compares centralized and distributed orchestration models, reviews state-of-the-art baseline solutions, and articulates the research gap addressed by our work.

2.1 Satellite Computing Paradigms

Space-based networking architectures have historically been categorized by their orbital altitudes: Geostationary Earth Orbit (GEO), Medium Earth Orbit (MEO), and Low Earth Orbit (LEO). GEO satellites, positioned at approximately 35,786 km, offer stationary coverage over vast geographic areas but suffer from high round-trip propagation delays (exceeding 250 ms) and high free-space path loss. MEO satellites (2,000 km – 35,786 km) reduce latency slightly but remain unsuitable for ultra-low-latency processing.

In contrast, LEO satellites operate at altitudes ranging from 300 km to 2,000 km. Their proximity to Earth drastically lowers propagation latency (10 – 40 ms round-trip) and reduces power loss, making them ideal for high-throughput, real-time communications. However, LEO satellites travel at extreme velocities relative to the surface (~ 7.5 km/s), completing an orbit in approximately 90 – 120 minutes. This rapid mobility results in dynamic topologies where individual satellites remain in line-of-sight with a fixed ground position for only a few minutes.

To process data efficiently within this dynamic environment, satellite network architectures have evolved through three primary paradigms:

1. **Transparent Bent-Pipe Relay:** Satellites act merely as transparent repeaters. Raw data captured by Earth Observation (EO) sensors or transmitted by Ground Users (GUs) is amplified and forwarded directly to Ground Stations (GSs). Computation occurs entirely in terrestrial cloud centers. This

model suffers from severe downlink bottlenecks and complete connectivity loss when flying over remote regions (e.g., oceans or polar zones) lacking GS infrastructure.

2. **Terrestrial Cloud Offloading with Inter-Satellite Routing:** Satellites utilize Inter-Satellite Links (ISLs) to route raw data across orbital planes until a satellite with an active GS connection is reached. While this mitigates local GS coverage gaps, the volume of raw data still strains optical and RF link bandwidth.
3. **Orbital Edge Computing (OEC) / Satellite Edge Computing (SEC):** Satellites are equipped with onboard processing units—termed Satellite Edge Nodes (SENs)—capable of executing compute-intensive algorithm payloads (e.g., AI inference, image compression, signal filtering) directly in orbit. Raw data is processed locally at the source node or offloaded across ISLs to adjacent SENs. Only refined, actionable insights or small computation results are downlinked to Earth, drastically reducing bandwidth consumption and end-to-end response times.

2.2 Task Offloading and Routing in LEO Networks

Task offloading in Orbital Edge Computing involves receiving task execution requests generated by Ground Users (GUs) or space-borne sensors and strategically allocating their computational execution across available SENs while routing payloads through Inter-Satellite Links (ISLs).

2.2.1 System Entities and Network Topology

An OEC network architecture consists of three core structural entities:

- **Ground Users (GUs) / Remote Sensors:** Distributed terrestrial entities that issue computational task requests r_i . Each request is characterized by a data payload size D_r^{data} (in bits), required CPU compute cycles W_r , and a strict operational deadline D_r (in seconds).
- **Satellite Edge Nodes (SENs):** LEO satellites equipped with CPU cores, memory modules, and onboard energy reserves (solar panels and battery banks). Each SEN s_j possesses a processing capacity f_j (cycles/second) and an operational energy budget E_j .
- **Inter-Satellite Links (ISLs):** High-speed communication channels connecting adjacent satellites. Intra-plane ISLs connect satellites within the same orbital plane (stable distance), whereas inter-plane ISLs connect satellites in adjacent planes (dynamically varying distance and pointing angles). Each ISL link (u, v) is constrained by bandwidth B_{uv} and propagation delay τ_{uv} .

2.2.2 Workload Profiles and Constraints

Workloads submitted to an OEC network are typically heterogeneous, categorized into two main profiles:

- **Real-Time Workloads:** Tasks generated by time-critical applications (e.g., emergency disaster detection, defense alerts) requiring immediate processing with tight deadlines D_r .
- **Batch / Delay-Tolerant Workloads:** Tasks associated with routine telemetry, scientific processing, or background data aggregation with relaxed deadlines.

Offloading decisions must jointly satisfy physical compute constraints ($\sum W_r \leq f_j$), network bandwidth limits, energy consumption limits (computation power P_{comp} and transmission power P_{tx}), and strict end-to-end deadline bounds ($T_{total} = T_{propagation} + T_{transmission} + T_{queue} + T_{computation} \leq D_r$).

2.3 Centralized vs. Distributed Orchestration

Managing resource allocation across hundreds of moving SENs requires an orchestration paradigm. Literature categorizes these paradigms into two primary architectures: Centralized and Distributed.

2.3.1 Centralized Orchestration

In a centralized orchestration architecture, a dedicated master entity—such as a designated lead satellite (Ground/Orbital Orchestrator) or a ground control station—collects global network state information (CPU load, battery state, link traffic) from all SENs.

- **Advantages:** The central orchestrator maintains complete global visibility over all network resources, enabling mathematically optimal decision-making. Global optimization models (e.g., Integer Linear Programming) can eliminate load imbalances, prevent local hotspots, and minimize network-wide energy consumption.
- **Disadvantages:** Centralized schemes incur significant control signaling overhead to aggregate state information. Furthermore, requests must travel to the orchestrator, causing queuing delays and introducing a single point of failure and bottleneck under high request arrival rates (λ).

2.3.2 Distributed Orchestration

In distributed orchestration, individual SENs or localized clusters make autonomous offloading and routing decisions based solely on locally observed state information or partial neighborhood telemetry.

- **Advantages:** Distributed schemes eliminate centralized control bottlenecks, offer sub-millisecond decision latency, and demonstrate high fault tolerance against dynamic topology changes and satellite failures.
- **Disadvantages:** Lacking global visibility, distributed algorithms often converge to sub-optimal solutions. They are prone to localized network congestion, unbalanced CPU utilization, and premature battery depletion on popular forwarding paths.

2.4 State-of-the-Art Approaches

To place our research in context, we review key baseline algorithms and mathematical models established in recent IEEE literature on Orbital Edge Computing and Task Offloading.

2.4.1 Distributed Heuristic Frameworks

Heuristic approaches sacrifice theoretical optimality in favor of computational speed and low decision latency:

- **OrbitAware Architecture:** A dynamic heuristic framework designed specifically for highly dynamic LEO topologies. *OrbitAware* leverages periodic orbital trajectory updates to predict link availability and select shortest processing/routing paths locally, reducing control overhead at the cost of potential link queue saturation.
- **DTS-base (Dynamic Task Scheduling - Base):** A greedy distributed scheduling baseline that assigns incoming tasks to the nearest available SEN with sufficient idle CPU capacity. While fast, it ignores global energy budgets and multi-hop link traffic.
- **DTS-APopt (Dynamic Task Scheduling - Access Point Optimized):** An enhanced version of *DTS-base* that incorporates multi-criteria decision metrics, optimizing both the local access point selection and initial ISL hop bandwidth before dispatching tasks.

2.4.2 Optimization-Based and Distributed ILP Formulations

To improve allocation quality, recent studies have formulated mathematical optimization models using Integer Linear Programming (ILP) and mixed-integer techniques:

- **Distributed / Hierarchical ILP:** Frameworks that decompose a global ILP problem into smaller sub-problems solved locally across satellite clusters. While more scalable than pure centralized ILP, distributed ILP approaches suffer from convergence delays and coordination overhead during inter-cluster task transfers.

- **Weighted Objective Functions:** Optimization models in the literature frequently balance competing goals—such as latency minimization versus energy preservation—by assigning scalar weights (W_{lat}, W_{en}) to objective terms.

2.5 Research Gap and Thesis Positioning

While state-of-the-art distributed heuristics offer rapid processing times and distributed ILP frameworks attempt local decomposition, a significant research gap remains regarding the practical limits of **Centralized ILP Orchestration** in LEO satellite networks.

Existing literature frequently assumes either ideal zero-latency centralized orchestrators or dismisses centralized optimization entirely due to its computational complexity. Few works rigorously examine the trade-offs when a centralized ILP orchestrator is equipped with an integrated **Dynamic Batching** and **Timeout Mechanism** (BS/BT) to manage request queues under heavy workloads ($\lambda > 6$ req/s).

This thesis addresses this exact gap by:

1. Formalizing a centralized ILP model that dynamically toggles between latency minimization (*Mapping Time*) and energy conservation (*Mapping Energy*).
2. Quantifying the systemic performance bounds of centralized optimization when coupled with dynamic request batching and queuing delay compensation.
3. Conducting a rigorous comparative benchmark against established state-of-the-art heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP variants under realistic discrete-event simulation conditions (*SimPy*).

Chapter 3

System Architecture and ILP Formulation

3.1 System Model and Assumptions

The system models a regional cluster within a LEO constellation. A designated *Master Satellite* acts as the regional coordinator, receiving incoming service requests $R = \{r_1, r_2, \dots, r_m\}$ and dispatching execution orders to local *Satellite Edge Nodes* (SENs) $S = \{1, 2, \dots, n\}$ via ISLs.

Each service request $r \in R$ is characterized by its input data size s_r , compute requirement c_r , maximum latency tolerance (deadline) D_r , and penalty P_r paid if rejected.

3.2 ILP Mathematical Formulation

3.2.1 Decision Variables and Objective Functions

Let binary variable $x_{r,i} \in \{0, 1\}$ indicate whether request r is assigned to SEN i . The objective function minimizes the normalized cost combining latency $\hat{R}_{r,i}$ and energy consumption $\hat{e}_{r,i}$:

$$\min Z = \sum_{r \in R} \left[\sum_{i \in S} x_{r,i} (w_{en} \hat{e}_{r,i} + w_{lat} \hat{R}_{r,i}) + P_r \left(1 - \sum_{i \in S} x_{r,i} \right) \right] \quad (3.1)$$

where w_{lat} and w_{en} are strategy weights:

- **Mapping Time:** $w_{lat} = 1, w_{en} = 0$, prioritizing minimal execution and routing latency.
- **Mapping Energy:** $w_{lat} = 0, w_{en} = 1$, prioritizing energy preservation across SEN nodes.

3.2.2 System Constraints

The optimization operates subject to the following linear constraints:

1. **Uniqueness Constraint:** Each request is assigned to at most one SEN:

$$\sum_{i \in S} x_{r,i} \leq 1, \quad \forall r \in R \quad (3.2)$$

2. **Energy Budget Constraint:** Total processing energy on SEN i cannot exceed its remaining battery capacity B_i :

$$\sum_{r \in R} x_{r,i} \cdot \epsilon_{i,r}^{cpu} \leq B_i, \quad \forall i \in S \quad (3.3)$$

3. **Service Deadline Constraint:**

$$x_{r,i} \cdot (t_{queue,r} + R_{r,i}) \leq D_r, \quad \forall r \in R, \forall i \in S \quad (3.4)$$

3.3 Dynamic Batching Mechanism

To prevent invoking the ILP solver for every single incoming request—which causes high computational overhead—the Master collects requests into batches[cite: 18].

- **Batch Size (BS):** Triggers solver execution once BS requests accumulate in the queue[cite: 18].
- **Batch Timeout (BT):** Forces batch resolution if BT seconds elapse before reaching BS requests, avoiding request starvation[cite: 18].

3.4 Timing and Queuing Delay Compensation

Since requests spend time $t_{queue,r}$ in the queue prior to ILP execution, the deadline constraint is dynamically adjusted before solving[cite: 18]. The remaining allowable service time for request r becomes $D'_r = D_r - t_{queue,r}$ [cite: 18]. If $t_{queue,r} \geq D_r$, the task is immediately dropped prior to optimization to avoid wasting solver resources[cite: 18, 20].

Chapter 4

Performance Analysis and Evaluation

4.1 Simulation Framework

The proposed orchestrator was implemented and evaluated using a discrete-event simulation environment built in Python with the `SimPy` framework and solved via PuLP / CBC solver[cite: 19].

4.2 Experimental Setup and Workload Profiles

Simulations were conducted under varying request arrival rates ($\lambda \in [2, 10]$ req/s)[cite: 19, 20]. Workloads consisted of heterogeneous service classes combining CPU-intensive tasks and data-intensive payloads.

4.3 Evaluation of Optimization Strategies

4.3.1 Mapping Time vs. Mapping Energy Comparison

Experimental results demonstrate that *Mapping Time* achieves lower average response times and higher success rates under delay-sensitive workloads. In contrast, *Mapping Energy* succeeds in extending node operational lifespans but increases task queuing delays, resulting in higher deadline expiration rates under elevated arrival rates.

4.3.2 Impact of Batching Parameters (BS and BT)

Small batch sizes ($BS = 5, BS = 10$) keep queuing delays low but increase total solver runtime[cite: 18]. Larger batch sizes ($BS = 20, BS = 30$) optimize solver throughput but require a fine-tuned Batch Timeout ($BT \approx 0.1 - 0.2$ s) to prevent timing violations[cite: 18].

4.4 Bottleneck and Collapse Analysis

4.4.1 System Behavior Under Heavy Load ($\lambda > 6$ req/s)

As arrival rate λ exceeds 6 req/s, system completion rate experiences a sharp drop[cite: 20].

4.4.2 Physical Saturation vs. Deadline Exceeded Failures

Detailed failure analysis reveals that system degradation is predominantly driven by *Deadline Exceeded* events during queue accumulation rather than physical CPU or memory resource saturation[cite: 20].

4.5 Sensitivity Analysis: Deadline Relaxation (ΔD)

Sensitivity experiments evaluating deadline relaxation (ΔD) showed that relaxing tight real-time constraints by a marginal offset significantly recovers completion rates, proving that queuing delay adaptation is the primary leverage point for centralized satellite orchestrators[cite: 21].

Chapter 5

Comparative Benchmark

5.1 Experimental Setup and Baseline Algorithms

To evaluate the trade-offs of centralized ILP orchestration, the system was benchmarked against four baseline algorithms from the state of the art[cite: 22]:

1. **OrbitAware:** Trajectory-aware predictive offloading[cite: 22].
2. **DTS-base:** Distance-based greedy heuristic ($O(n \log n)$ complexity)[cite: 22].
3. **DTS-APopt:** Distance-based offloading with local access-point optimization[cite: 22].
4. **Distributed ILP:** Decentralized exact optimization solved locally per orbital sector[cite: 22].

5.2 Comparative Performance Evaluation

- **Task Completion Rate (Success Rate):** Under light to moderate traffic ($\lambda \leq 4$ req/s), Centralized ILP achieves superior completion rates due to global resource awareness[cite: 23].
- **Response Latency:** Distributed heuristics (DTS-base, DTS-APopt) achieve significantly lower initial response latency due to immediate decision execution without batch queuing delays[cite: 23].
- **Load Balancing Degree:** Centralized ILP yields a superior load balancing distribution across SENSs compared to greedy heuristics[cite: 23].

5.3 Trade-Off Analysis

The evaluation quantifies a fundamental architectural trade-off[cite: 23]:

- *Centralized ILP* offers theoretical global optimality and optimal energy-latency balance but suffers from queuing latency under heavy loads[cite: 23].
- *Distributed heuristics* sacrifice global optimality but offer superior responsiveness and scalability in dynamic LEO satellite environments[cite: 23].

Chapter 6

Conclusions and Future Work

6.1 Summary of Findings

This thesis presented the design, implementation, and performance evaluation of a centralized ILP orchestrator for task routing in LEO satellite networks[cite: 17]. By incorporating a dynamic batching mechanism (BS, BT) and queue-aware timing compensation, the orchestrator mitigates solver invocation overheads and request starvation[cite: 18]. Simulations confirmed that under heavy load ($\lambda > 6$ req/s), performance collapse is driven by queuing timeout deadlines rather than physical hardware limits[cite: 20].

6.2 Limitations of the Centralized Approach

The primary limitation of centralized ILP lies in its single-point decision bottleneck at the Master satellite, which introduces queuing latency during traffic spikes and depends heavily on reliable ISL control signaling[cite: 18, 23].

6.3 Future Research Directions

Future work will explore:

- **Hybrid Orchestration:** Combining centralized global optimization with local heuristic fallback during traffic surges.
- **Adaptive Machine Learning Batching:** Dynamically adjusting BS and BT parameters in real-time based on observed arrival rate fluctuations (λ).

Bibliography

- [1] BOYD, S. AND VANDENBERGHE, L. *Convex Optimization*. Cambridge University Press, Cambridge, UK (2004).
- [2] CASALICCHIO, E. ET AL. Handling of energy- and time-constrained heterogeneous satellite computing services. In *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*, pp. 1–12. IEEE (2025).
- [3] DENBY, B. AND LUCIA, B. Orbital edge computing: Nanosatellite clusters as accelerators for earth observation. *IEEE Micro*, **40** (2019), 13.
- [4] FAN, W., MENG, Q., WANG, G., BIAN, H., AND LIU, Y. Satellite edge intelligence: Drl-based resource management for task inference in leo-based satellite-ground collaborative networks. *IEEE Transactions on Mobile Computing*, **24** (2025), 10710.
- [5] GUROBI OPTIMIZATION, LLC. *Gurobi Optimizer Reference Manual* (2024). Available from: <https://www.gurobi.com>.
- [6] NEUMANN, A., AICHELE, C., LINDNER, M., AND FUX, S. Better approach to mobile ad-hoc networking (b.a.t.m.a.n.). Tech. rep., IETF Draft (2008). Draft-wunderlich-openmesh-manet-routing-00.
- [7] SCHERFKE, S. AND LÜNSDORF, O. *SimPy: Discrete Event Simulation for Python* (2023). Version 4.1.1. Available from: <https://simpy.readthedocs.io/>.
- [8] TANG, X. AND YI, X. A general metric of load balancing in δ -range. In *Advanced Parallel Processing Technologies*, pp. 311–321. Springer (2003).
- [9] WANG, S., ZHANG, Q., ZHAO, C., ET AL. Satellite computing: Vision and challenges – tiansuan constellation. *IEEE Internet of Things Journal*, **10** (2023), 17550.
- [10] WOLSEY, L. A. *Integer Programming*. John Wiley & Sons, New York, NY, USA (1998).
- [11] ZHANG, Y., LI, X., ZHOU, H., AND CHEN, D. A comprehensive survey on orbital edge computing: Systems, applications, and algorithms. *IEEE Communications Surveys & Tutorials*, **25** (2023), 2100.

-
- [12] ZHONG, L., LI, Y., GE, M.-F., FENG, M., AND MAO, S. Joint task offloading and resource allocation for leo satellite-based mobile edge computing systems with heterogeneous task demands. *IEEE Transactions on Vehicular Technology*, **74** (2025), 11337.